

Payments — Wallet, Razorpay, Stripe, Payouts & Invoices

App: apps/payments **Gateways:** Razorpay (India), Stripe (International) **Key principle:** b-secure never stores raw card/bank data. All sensitive data stays at the payment gateway.

Table of Contents

1. [Payment Architecture](#)
 2. [Wallet System](#)
 3. [Top-up Flow \(Razorpay\)](#)
 4. [Top-up Flow \(Stripe\)](#)
 5. [Booking Payment Flow](#)
 6. [Webhook Handling](#)
 7. [Refund Flow](#)
 8. [Guard Payout System](#)
 9. [Invoice Generation \(PDF\)](#)
 10. [Payment Service Layer](#)
 11. [Pricing Engine](#)
-

1. Payment Architecture

User tops up wallet:

```
App → POST /api/payments/wallet/topup/initiate/  
    ← Gateway order ID  
App → Opens Razorpay/Stripe SDK with order ID  
User completes payment in gateway UI  
Gateway → POST /api/payments/webhook/razorpay/ (server-to-server)  
    → Wallet credited  
    → User notified
```

Booking payment:

```
On session completion:  
Celery task (process_session_payment)  
    → Deduct from user wallet  
    → Record transaction
```

- Update booking payment status
- Schedule guard earnings credit

Guard payout:

Nightly/Weekly Celery beat task

- Aggregate guard earnings
- Create Payout record
- Admin approves (or auto-approve below threshold)
- Razorpay Payouts API → Bank/UPI transfer

2. Wallet System

apps/payments/services.py

```
from decimal import Decimal
from django.db import transaction as db_transaction
from django.utils import timezone
from .models import Wallet, Transaction
```

class WalletService:

```
    @classmethod
    @db_transaction.atomic
    def credit(cls, user, amount: Decimal, transaction_type: str,
              booking=None, description: str = '', admin_note:
                str = '') -> Transaction:
        """
        Add funds to user wallet.
        Uses SELECT FOR UPDATE to prevent race conditions.
        """
        wallet = Wallet.objects.select_for_update().get(user=user)
        balance_before = wallet.balance
        wallet.balance += amount
        wallet.save(update_fields=['balance', 'updated_at'])

        return Transaction.objects.create(
            wallet=wallet,
            transaction_type=transaction_type,
            amount=amount,
            balance_before=balance_before,
            balance_after=wallet.balance,
            status='SUCCESS',
            booking=booking,
            description=description,
            admin_note=admin_note,
            gateway='INTERNAL',
        )

    @classmethod
```

```

@db_transaction.atomic
def debit(cls, user, amount: Decimal, transaction_type: str,
          booking=None, description: str = '') -> Transaction:
    """
    Deduct funds from user wallet.
    Raises InsufficientBalanceError if balance too low.
    """
    wallet = Wallet.objects.select_for_update().get(user=user)

    if wallet.available_balance < amount:
        raise InsufficientBalanceError(
            f'Wallet balance (₹{wallet.available_balance}) is
            insufficient for ₹{amount}.'
        )

    balance_before = wallet.balance
    wallet.balance -= amount
    wallet.save(update_fields=['balance', 'updated_at'])

    return Transaction.objects.create(
        wallet=wallet,
        transaction_type=transaction_type,
        amount=amount,
        balance_before=balance_before,
        balance_after=wallet.balance,
        status='SUCCESS',
        booking=booking,
        description=description,
        gateway='INTERNAL',
    )

@classmethod
@db_transaction.atomic
def lock_for_booking(cls, user, amount: Decimal, booking) ->
    Transaction:
    """
    Lock amount in wallet when booking is accepted.
    Locked funds cannot be used for other bookings.
    Released on cancellation, debited on completion.
    """
    wallet = Wallet.objects.select_for_update().get(user=user)

    if wallet.available_balance < amount:
        raise InsufficientBalanceError(
            f'Insufficient balance to lock ₹{amount} for this
            booking.'
        )

    balance_before = wallet.balance
    wallet.locked_balance += amount
    wallet.save(update_fields=['locked_balance',
                              'updated_at'])

```

```

    return Transaction.objects.create(
        wallet=wallet,
        transaction_type='BOOKING_LOCK',
        amount=amount,
        balance_before=balance_before,
        balance_after=wallet.balance,
        status='SUCCESS',
        booking=booking,
        description=f'Funds locked for booking
#{str(booking.id)[:8]}'
    )

    @classmethod
    @db_transaction.atomic
    def release_lock(cls, user, amount: Decimal, booking) ->
    Transaction:
        """Release locked funds back to available balance (on
        cancellation)."""
        wallet = Wallet.objects.select_for_update().get(user=user)
        balance_before = wallet.balance
        wallet.locked_balance = max(Decimal('0'),
        wallet.locked_balance - amount)
        wallet.save(update_fields=['locked_balance',
        'updated_at'])

    return Transaction.objects.create(
        wallet=wallet,
        transaction_type='BOOKING_UNLOCK',
        amount=amount,
        balance_before=balance_before,
        balance_after=wallet.balance,
        status='SUCCESS',
        booking=booking,
        description='Booking cancelled – funds released',
    )

class InsufficientBalanceError(Exception):
    pass

```

3. Top-up Flow (Razorpay)

```
# apps/payments/services.py
```

```

import razorpay
from django.conf import settings
from decimal import Decimal
import uuid

```

```
class RazorpayService:
```

```
_client = None
```

```
@classmethod
```

```
def get_client(cls):  
    if cls._client is None:  
        cls._client = razorpay.Client(  
            auth=(settings.RAZORPAY_KEY_ID,  
                settings.RAZORPAY_KEY_SECRET)  
        )  
    return cls._client
```

```
@classmethod
```

```
def create_topup_order(cls, user, amount_inr: Decimal) ->  
    dict:  
    """  
    Create a Razorpay order for wallet top-up.  
    Returns data needed by mobile SDK to open checkout.  
    """
```

```
from .models import PaymentOrder
```

```
amount_paise = int(amount_inr * 100) # Razorpay uses  
smallest currency unit (paise)
```

```
# Create order at Razorpay
```

```
rz_order = cls.get_client().order.create({  
    'amount': amount_paise,  
    'currency': 'INR',  
    'receipt': f'topup_{uuid.uuid4().hex[:16]}',  
    'notes': {  
        'user_id': str(user.id),  
        'purpose': 'wallet_topup',  
    }  
})
```

```
# Record in our DB
```

```
payment_order = PaymentOrder.objects.create(  
    user=user,  
    amount=amount_inr,  
    currency='INR',  
    purpose='WALLET_TOPUP',  
    gateway='RAZORPAY',  
    gateway_order_id=rz_order['id'],  
    gateway_response=rz_order,  
)
```

```
return {  
    'order_id': str(payment_order.id),  
    'gateway_order_id': rz_order['id'],  
    'gateway': 'RAZORPAY',  
    'amount': float(amount_inr),  
    'currency': 'INR',  
    'razorpay_key_id': settings.RAZORPAY_KEY_ID,
```

```
}
```

```
@classmethod
```

```
def verify_payment_signature(cls, order_id: str, payment_id: str, signature: str) -> bool:
```

```
    """Verify HMAC signature after client-side payment completion."""
```

```
    import hmac
```

```
    import hashlib
```

```
    generated = hmac.new(
        settings.RAZORPAY_KEY_SECRET.encode(),
        f'{order_id}|{payment_id}'.encode(),
        hashlib.sha256
    ).hexdigest()
```

```
    return hmac.compare_digest(generated, signature)
```

```
@classmethod
```

```
def process_topup_confirmation(cls, gateway_order_id: str,
                               gateway_payment_id: str,
                               gateway_signature: str) ->
```

```
    dict:
```

```
    """
```

```
    Called by client after completing payment in Razorpay SDK.
    Verifies signature, credits wallet.
```

```
    """
```

```
    from .models import PaymentOrder
```

```
    from django.db import transaction as db_transaction
```

```
    # Verify signature
```

```
    if not cls.verify_payment_signature(
        gateway_order_id, gateway_payment_id,
        gateway_signature
    ):
```

```
        raise ValueError('SIGNATURE_MISMATCH')
```

```
    with db_transaction.atomic():
```

```
        order = PaymentOrder.objects.select_for_update().get(
            gateway_order_id=gateway_order_id
        )
```

```
        if order.status == 'PAID':
```

```
            # Idempotent – already processed
```

```
            return {'success': True, 'already_processed':
```

```
True}
```

```
        order.status = 'PAID'
```

```
        order.gateway_response = {
```

```
            'payment_id': gateway_payment_id,
```

```
            'signature': gateway_signature,
```

```
        }
```

```
        order.save()
```

```

# Credit wallet
txn = WalletService.credit(
    user=order.user,
    amount=order.amount,
    transaction_type='TOPUP',
    description=f'Wallet top-up via Razorpay',
    gateway='RAZORPAY',
)
# Add gateway reference to transaction
Transaction.objects.filter(id=txn.id).update(
    gateway='RAZORPAY',
    gateway_order_id=gateway_order_id,
    gateway_payment_id=gateway_payment_id,
)

# Notify user
from apps.notifications.tasks import
send_wallet_topup_notification
send_wallet_topup_notification.delay(str(order.user_id),
float(order.amount))

return {
    'success': True,
    'new_balance': float(order.user.wallet.balance),
}

```

4. Top-up Flow (Stripe)

apps/payments/services.py (continued)

```

import stripe
from django.conf import settings

class StripeService:

    @classmethod
    def create_payment_intent(cls, user, amount_inr: Decimal) ->
    dict:
        """Create Stripe PaymentIntent for wallet top-up."""
        from .models import PaymentOrder

        stripe.api_key = settings.STRIPE_SECRET_KEY

        # Convert INR to paise (Stripe uses smallest currency
        unit)
        amount_paise = int(amount_inr * 100)

        intent = stripe.PaymentIntent.create(
            amount=amount_paise,
            currency='inr',

```

```

        metadata={
            'user_id': str(user.id),
            'purpose': 'wallet_topup',
        },
    )

    PaymentOrder.objects.create(
        user=user,
        amount=amount_inr,
        currency='INR',
        purpose='WALLET_TOPUP',
        gateway='STRIPE',
        gateway_order_id=intent['id'],
        gateway_response=intent,
    )

    return {
        'client_secret': intent['client_secret'],
        'publishable_key': settings.STRIPE_PUBLISHABLE_KEY,
        'gateway': 'STRIPE',
    }

```

5. Booking Payment Flow

apps/payments/tasks.py

```

from celery import shared_task
from decimal import Decimal
import logging

```

```

logger = logging.getLogger(__name__)

```

```

@shared_task(
    bind=True,
    max_retries=3,
    default_retry_delay=60,
    queue='default',
    name='payments.process_session_payment'
)
def process_session_payment(self, booking_id: str):
    """

```

*Process payment after a session is completed.
Called by the booking completion signal/service.*

Steps:

- 1. Calculate final amount (actual duration × rate)*
- 2. Apply any promo discounts*
- 3. Debit user wallet (already locked – convert lock to debit)*
- 4. Calculate platform fee and guard earnings*

```

5. Update booking financial fields
6. Generate invoice async
7. Notify user (payment receipt)
"""

from apps.bookings.models import Booking
from apps.payments.services import WalletService
from apps.payments.pricing import PricingEngine
from django.db import transaction as db_transaction

try:
    booking = Booking.objects.select_related('user',
        'guard').get(id=booking_id)

    with db_transaction.atomic():
        # Calculate actual amount (based on actual session
        duration)
        pricing = PricingEngine.calculate_final_price(booking)

        # Convert locked amount → actual debit
        WalletService.release_lock(booking.user,
            pricing['locked_amount'], booking)
        WalletService.debit(
            user=booking.user,
            amount=pricing['total_amount'],
            transaction_type='BOOKING_DEBIT',
            booking=booking,
            description=f'Payment for {booking.service_type}
session',
        )

        # Update booking
        Booking.objects.filter(id=booking_id).update(
            total_amount=pricing['total_amount'],
            platform_fee=pricing['platform_fee'],
            guard_earnings=pricing['guard_earnings'],
            tax_amount=pricing['tax_amount'],
        )

        # Async: generate PDF invoice + notify user
        generate_invoice_pdf.delay(booking_id)
        from apps.notifications.tasks import send_payment_receipt
        send_payment_receipt.delay(booking_id)

except Exception as exc:
    logger.error(f'process_session_payment failed for
        {booking_id}: {exc}')
    raise self.retry(exc=exc)

```

6. Webhook Handling

```
# apps/payments/webhooks.py
```

```
import hmac, hashlib, json, logging
from django.conf import settings
from django.views.decorators.csrf import csrf_exempt
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import AllowAny
from rest_framework.response import Response
```

```
from .tasks import process_razorpay_event, process_stripe_event
```

```
logger = logging.getLogger(__name__)
```

```
@api_view(['POST'])
@permission_classes([AllowAny])
@csrf_exempt
def razorpay_webhook(request):
    """
    POST /api/payments/webhook/razorpay/
    Handles: payment.captured, payment.failed, payout.processed,
             payout.failed
    """
    # Verify HMAC signature
    signature = request.headers.get('X-Razorpay-Signature', '')
    payload_bytes = request.body
    secret = settings.RAZORPAY_WEBHOOK_SECRET.encode('utf-8')
    expected_sig = hmac.new(secret, payload_bytes,
                             hashlib.sha256).hexdigest()

    if not hmac.compare_digest(expected_sig, signature):
        logger.warning('Razorpay webhook: invalid signature')
        return Response({'error': 'Invalid signature'},
                        status=400)

    event = request.data
    event_id = event.get('payload', {}).get('payment',
                                             {}).get('entity', {}).get('id', 'unknown')

    logger.info(f'Razorpay webhook received: {event.get("event")} '
               f'id={event_id}')

    # Dispatch to Celery (async processing, idempotent)
    process_razorpay_event.apply_async(
        args=[event],
        queue='high_priority',
        task_id=f'rz_webhook_{event_id}', # Idempotency key
    )

    # Always return 200 immediately – Razorpay retries if non-200
    return Response({'status': 'received'}, status=200)
```

```

@shared_task(bind=True, max_retries=3, queue='default')
def process_razorpay_event(self, event: dict):
    """
    Idempotent handler for Razorpay webhook events.
    """
    event_type = event.get('event')
    payload = event.get('payload', {})

    handlers = {
        'payment.captured': _handle_payment_captured,
        'payment.failed': _handle_payment_failed,
        'payout.processed': _handle_payout_processed,
        'payout.failed': _handle_payout_failed,
    }

    handler = handlers.get(event_type)
    if handler:
        handler(payload)
    else:
        logger.info(f'Razorpay webhook: unhandled event type {event_type}')

def _handle_payment_captured(payload: dict):
    """Wallet top-up completed via Razorpay."""
    from apps.payments.models import PaymentOrder, Transaction
    from apps.payments.services import WalletService
    from django.db import transaction as db_transaction

    payment = payload.get('payment', {}).get('entity', {})
    rz_order_id = payment.get('order_id')
    rz_payment_id = payment.get('id')
    amount_paise = payment.get('amount', 0)
    amount_inr = Decimal(str(amount_paise)) / 100

    with db_transaction.atomic():
        try:
            order = PaymentOrder.objects.select_for_update().get(
                gateway_order_id=rz_order_id,
                gateway='RAZORPAY'
            )
        except PaymentOrder.DoesNotExist:
            logger.error(f'PaymentOrder not found for rz_order_id={rz_order_id}')
            return

        if order.status == 'PAID':
            logger.info(f'Razorpay payment {rz_payment_id} already processed, skipping')
            return

```

```

order.status = 'PAID'
order.save()

WalletService.credit(
    user=order.user,
    amount=amount_inr,
    transaction_type='TOPUP',
    description='Wallet top-up (Razorpay webhook)',
)

from apps.notifications.tasks import
    send_wallet_topup_notification
send_wallet_topup_notification.delay(str(order.user_id),
    float(amount_inr))

```

7. Refund Flow

apps/payments/services.py

class RefundService:

```

REFUND_POLICY = {
    # minutes_before_start: refund_percentage
    60: 100,    # >60 min before: full refund
    30: 100,    # 30-60 min: full refund
    10: 50,     # 10-30 min: 50% refund
    0: 0,       # <10 min or after arrival: no refund
}

@classmethod
def calculate_refund_amount(cls, booking) -> Decimal:
    from django.utils import timezone
    from decimal import Decimal

    if booking.status in ('REQUESTED', 'BROADCAST'):
        return booking.total_amount or Decimal('0') # Full
        refund before guard assigned

    if not booking.total_amount:
        return Decimal('0')

    if booking.guard_arrived_at:
        return Decimal('0') # Guard arrived - no refund

    minutes_until_start = (
        booking.scheduled_start - timezone.now()
    ).total_seconds() / 60

    if minutes_until_start > 30:
        return booking.total_amount
    elif minutes_until_start > 10:

```

```

        return booking.total_amount * Decimal('0.5')
    else:
        return Decimal('0')

@classmethod
def process_refund(cls, booking, reason: str = '') -> dict:
    """Issue refund to user wallet after cancellation."""
    refund_amount = cls.calculate_refund_amount(booking)

    if refund_amount > 0:
        WalletService.credit(
            user=booking.user,
            amount=refund_amount,
            transaction_type='REFUND',
            booking=booking,
            description=f'Refund for cancelled booking.
Reason: {reason}',
        )

    return {
        'refund_amount': float(refund_amount),
        'refund_percentage': float(
            refund_amount / booking.total_amount * 100) if
booking.total_amount else 0
        ),
    }

```

8. Guard Payout System

apps/payments/tasks.py

```

from celery import shared_task
from decimal import Decimal
from django.utils import timezone
from datetime import timedelta

@shared_task(queue='scheduled',
             name='payments.process_weekly_payouts')
def process_weekly_payouts():
    """
    Runs every Friday at 10 PM IST (configured in Celery beat).
    Aggregates completed session earnings for all guards
    and creates Payout records for admin approval.
    """

    from apps.bookings.models import Booking
    from apps.guards.models import GuardProfile
    from apps.payments.models import Payout
    from django.db.models import Sum

    period_end = timezone.now().date()

```

```

period_start = period_end - timedelta(days=7)

# Find all guards with earnings in this period
guards_with_earnings = Booking.objects.filter(
    status='COMPLETED',
    session_ended_at__date__gte=period_start,
    session_ended_at__date__lte=period_end,
    guard_earnings__isnull=False,
).values('guard').annotate(
    total_earnings=Sum('guard_earnings')
).filter(
    total_earnings__gt=200 # Minimum payout threshold ₹200
)

payouts_created = 0
for item in guards_with_earnings:
    guard = GuardProfile.objects.get(id=item['guard'])

    # Check if payout already exists for this period (idempotency)
    existing = Payout.objects.filter(
        guard=guard,
        period_start=period_start,
        period_end=period_end,
    ).exists()

    if not existing:
        payout = Payout.objects.create(
            guard=guard,
            amount=item['total_earnings'],
            period_start=period_start,
            period_end=period_end,
            status='PENDING',
        )
        # Link sessions to this payout
        sessions = Booking.objects.filter(
            guard=guard,
            status='COMPLETED',
            session_ended_at__date__gte=period_start,
            session_ended_at__date__lte=period_end,
        )
        payout.bookings.set(sessions)
        payouts_created += 1

    return f'Created {payouts_created} payouts for period {period_start} to {period_end}'

```

```

@shared_task(queue='default', name='payments.execute_payout')
def execute_payout(payout_id: str):
    """

```

Triggered by admin approval of a payout.

*Calls Razorpay Payouts API to transfer money to guard's bank/
UPI.*

"""

```
import razorpay
from apps.payments.models import Payout
from django.conf import settings
```

```
payout =
    Payout.objects.select_related('guard').get(id=payout_id)
```

```
if payout.status != 'PENDING':
    return # Already processed
```

```
client = razorpay.Client(
    auth=(settings.RAZORPAY_KEY_ID,
        settings.RAZORPAY_KEY_SECRET)
)
```

```
guard = payout.guard
```

Build payout request based on guard's preference

```
if guard.payout_preference == 'UPI' and guard.upi_id:
```

```
    fund_account = {
        'account_type': 'vpa',
        'vpa': {'address': guard.upi_id},
    }
```

```
else:
```

```
    fund_account = {
        'account_type': 'bank_account',
        'bank_account': {
            'name': guard.user.full_name,
            'ifsc': guard.bank_ifsc_code,
            'account_number': guard.bank_account_number,
        }
    }
```

```
try:
```

```
    rz_payout = client.payout.create({
        'account_number': settings.RAZORPAY_SOURCE_ACCOUNT,
        'amount': int(payout.amount * 100), # Paise
        'currency': 'INR',
        'mode': 'UPI' if guard.payout_preference == 'UPI'
        else 'IMPS',
        'purpose': 'payout',
        'fund_account': fund_account,
        'notes': {
            'guard_id': str(guard.id),
            'payout_id': str(payout.id),
            'period': f'{payout.period_start} to
{payout.period_end}',
        }
    })
```

```

Payout.objects.filter(id=payout_id).update(
    status='PROCESSING',
    razorpay_payout_id=rz_payout['id'],
)

except Exception as e:
    Payout.objects.filter(id=payout_id).update(
        status='FAILED',
        failure_reason=str(e),
    )
    raise

```

9. Invoice Generation (PDF)

apps/payments/tasks.py

```

@shared_task(queue='low_priority',
              name='payments.generate_invoice_pdf')
def generate_invoice_pdf(booking_id: str):
    """
    Generate PDF invoice for a completed booking and store on S3.
    Called after session payment is processed.
    """

    from apps.bookings.models import Booking
    from weasyprint import HTML, CSS
    from django.template.loader import render_to_string
    import boto3
    from django.conf import settings
    import io

    booking = Booking.objects.select_related(
        'user', 'guard__user'
    ).get(id=booking_id)

    # Render HTML template
    html_content = render_to_string('payments/invoice.html', {
        'booking': booking,
        'invoice_number': f'BSE--{str(booking.id)[:8].upper()}',
        'issued_at': booking.session_ended_at,
        'company_name': 'b-secure',
        'gst_in': settings.COMPANY_GSTIN,
        'tax_rate': 18,
    })

    # Convert to PDF
    pdf_buffer = io.BytesIO()
    HTML(string=html_content).write_pdf(
        pdf_buffer,
        stylesheets=[CSS(filename='static/css/invoice.css')]
    )

```

```

pdf_buffer.seek(0)

# Upload to S3 (private)
s3 = boto3.client('s3')
s3_key = f'invoices/{booking.user_id}/{booking_id}.pdf'
s3.upload_fileobj(
    pdf_buffer,
    settings.AWS_STORAGE_BUCKET_NAME,
    s3_key,
    ExtraArgs={'ContentType': 'application/pdf'}
)

# Store key on booking for later retrieval
from apps.bookings.models import Booking
Booking.objects.filter(id=booking_id).update(invoice_s3_key=s3_key)

# Send email with PDF attached
from apps.notifications.tasks import send_invoice_email
send_invoice_email.delay(booking_id, s3_key)

```

Invoice HTML Template: templates/payments/invoice.html

```

<!DOCTYPE html>
<html>
<head><title>Invoice - b-secure</title></head>
<body>
    <div class="header">
        <h1>b-secure</h1>
        <p>Invoice #{{ invoice_number }}</p>
        <p>Date: {{ issued_at|date:"d M Y" }}</p>
    </div>

    <div class="parties">
        <div class="bill-to">
            <h3>Billed To</h3>
            <p>{{ booking.user.full_name }}</p>
            <p>{{ booking.user.phone_number }}</p>
        </div>
    </div>

    <table class="items">
        <tr>
            <th>Description</th>
            <th>Duration</th>
            <th>Rate</th>
            <th>Amount</th>
        </tr>
        <tr>
            <td>{{ booking.get_service_type_display }} -
            {{ booking.guard_type_requested }}</td>
            <td>{{ booking.duration_display }}</td>
            <td>₹{{ booking.base_rate_per_hour }}/hr</td>
            <td>₹{{ booking.subtotal }}</td>
        </tr>
    </table>

```

```

        </tr>
        <tr>
            <td colspan="3">GST (18%)</td>
            <td>₹{{ booking.tax_amount }}</td>
        </tr>
        <tr class="total">
            <td colspan="3"><strong>Total</strong></td>
            <td><strong>₹{{ booking.total_amount }}</strong></td>
        </tr>
    </table>

    <div class="footer">
        <p>GSTIN: {{ gstin }}</p>
        <p>Thank you for choosing b-secure.</p>
    </div>
</body>
</html>

```

10. Payment Service Layer

apps/payments/views.py

```

from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from .services import RazorpayService, StripeService
from .serializers import TopupInitiateSerializer,
    TopupConfirmSerializer
from utils.storage import generate_presigned_url

class WalletTopupInitiateView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        serializer = TopupInitiateSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        amount = serializer.validated_data['amount']
        gateway = serializer.validated_data['gateway']

        if gateway == 'RAZORPAY':
            data =
            RazorpayService.create_topup_order(request.user, amount)
        else:
            data =
            StripeService.create_payment_intent(request.user, amount)

        return Response({'data': data})

```

```

class InvoiceDownloadView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request, booking_id):
        from apps.bookings.models import Booking
        try:
            booking = Booking.objects.get(id=booking_id,
            user=request.user, status='COMPLETED')
        except Booking.DoesNotExist:
            return Response({'error': {'code': 'NOT_FOUND'}},
            status=404)

        if not booking.invoice_s3_key:
            return Response({'error': {'code':
            'INVOICE_NOT_READY',
                                     'message': 'Invoice is
            being generated. Try again in a moment.'}}, status=202)

        url = generate_presigned_url(booking.invoice_s3_key,
        expiry_seconds=300)
        return Response({'data': {'download_url': url,
        'expires_in': 300}})

```

11. Pricing Engine

apps/payments/pricing.py

```

from decimal import Decimal, ROUND_HALF_UP
from django.conf import settings

```

Base rates (INR per hour) – configurable in admin

```

BASE_RATES = {
    'UNARMED': Decimal('150'),
    'ARMED': Decimal('300'),
    'FEMALE': Decimal('175'),
    'CPO': Decimal('600'),
    'EVENT': Decimal('200'),
}

```

```

PLATFORM_COMMISSION_RATE = Decimal('0.18')    # 18% platform fee
GST_RATE = Decimal('0.18')
            # 18% GST on platform fee

```

```

class PricingEngine:

```

```

    @classmethod

```

```

    def estimate_price(cls, guard_type: str, scheduled_start,
        scheduled_end,

```

```

        surge_multiplier: Decimal =
Decimal('1.0')) -> dict:
"""Estimate price for a booking before creation."""
duration_hours = cls._get_duration_hours(scheduled_start,
scheduled_end)
base_rate = BASE_RATES.get(guard_type,
BASE_RATES['UNARMED'])

subtotal = base_rate * Decimal(str(duration_hours)) *
surge_multiplier
platform_fee = (subtotal *
PLATFORM_COMMISSION_RATE).quantize(Decimal('0.01'),
ROUND_HALF_UP)
tax = (platform_fee * GST_RATE).quantize(Decimal('0.01'),
ROUND_HALF_UP)
total = (subtotal + tax).quantize(Decimal('0.01'),
ROUND_HALF_UP)
guard_earnings = (subtotal -
platform_fee).quantize(Decimal('0.01'), ROUND_HALF_UP)

return {
    'base_rate_per_hour': float(base_rate),
    'duration_hours': duration_hours,
    'surge_multiplier': float(surge_multiplier),
    'subtotal': float(subtotal),
    'platform_fee': float(platform_fee),
    'tax_amount': float(tax),
    'total_amount': float(total),
    'guard_earnings': float(guard_earnings),
}

@classmethod
def calculate_final_price(cls, booking) -> dict:
    """
    Calculate final price based on ACTUAL session duration.
    May differ from estimate if session ran over/under.
    """
    if booking.session_started_at and
booking.session_ended_at:
        actual_start = booking.session_started_at
        actual_end = booking.session_ended_at
    else:
        actual_start = booking.scheduled_start
        actual_end = booking.scheduled_end

    return cls.estimate_price(
        guard_type=booking.guard_type_requested,
        scheduled_start=actual_start,
        scheduled_end=actual_end,
        surge_multiplier=booking.surge_multiplier,
    )

@staticmethod
def _get_duration_hours(start, end) -> float:

```

```
delta = end - start
hours = delta.total_seconds() / 3600
return max(hours, 2.0) # Minimum 2 hours
```